Ain Shams University — Faculty of Engineering

CSE343: Web Development

Lab 3 Report

Interactive Calculator Webpage (JavaScript)

Student Name: Mahmoud Badwy Abdelhamied

Student ID: 24P0397

GitHub Repository: https://github.com/mahmoud-badwy/web-development-course-at-university/tree/main/lab3task

1. Objective

The task of Lab 3 is to build an interactive calculator webpage that performs basic mathematical operations. The page is split into three files: HTML for the structure, CSS for the design, and JavaScript for the behaviour. The design follows the iOS calculator shown in the lab.

2. GitHub Repository

The full source code is available at: https://github.com/mahmoud-badwy/web-development-course-at-university/tree/main/lab3task

File	Purpose
index.html	Page structure: the screen and the 19 buttons
style.css	iOS-style design and layout
script.js	Calculator logic and event handling
README.md	Project description and how to run it

3. Implementation

As suggested in the lab hints, the JavaScript is broken into three parts: DOM, Events and Logic. The code is kept as simple as possible and only uses concepts covered in Section 2 (HTML & CSS) and Section 3 (JavaScript).

3.1 HTML

The calculator is a container with a title, a screen and five rows of buttons. The screen is a paragraph with id="display" that starts at 0. Each row is a holding its buttons, and all the rows are inside one . The CSS and JavaScript are linked as external files using <link> and <script src>.

3.2 CSS

- **Selectors:** type selectors style body and h1, class selectors (.calculator, .key, .light, .operator, .zero) style the box and buttons, and an ID selector (#display) styles the screen.
- **Box model:** padding and margin space out the calculator and the buttons, and border-radius rounds the corners. margin: 40px auto centres the calculator.
- **Display property:** every button is display: inline-block with a fixed width and height, so four buttons sit side by side in each row. border-radius: 50% makes them round, and the 0 button is wider.
- **Colours:** dark grey number keys, light grey AC and orange operators, like the iOS calculator.

3.3 JavaScript

- **DOM:** the screen and the buttons container are selected with document.getElementById.
- **Events:** one click listener is attached to the buttons container instead of 19 separate listeners (event delegation). It checks e.target.nodeName to make sure a button was clicked, then reads the button's text with e.target.textContent.
- **Logic:** three variables remember the state: firstNumber, operator, and newNumber (whether the next digit starts a new number). An if / else if chain decides what to do with each button: AC clears everything, an operator saves the first number, = calculates the result, % divides by 100, and a digit is added to the screen.

- **calculate():** a small function uses a switch statement to add, subtract, multiply or divide two numbers. Number() turns the screen text into a number before doing the maths.

3.4 Concepts Used

Lab concept	Where it is used
External CSS and JS	<link href="style.css"> and <script src="script.js">
getElementById / textContent	Reading and updating the screen
addEventListener + delegation	One click listener on the buttons container
Event object	e.target, e.target.nodeName, e.target.textContent
Variables (let / const)	firstNumber, operator, newNumber, value
Conditionals	if / else if chain to handle each button
Functions and switch	calculate() picks the right operation
Display: inline-block	Placing four buttons side by side in each row

4. Screenshots

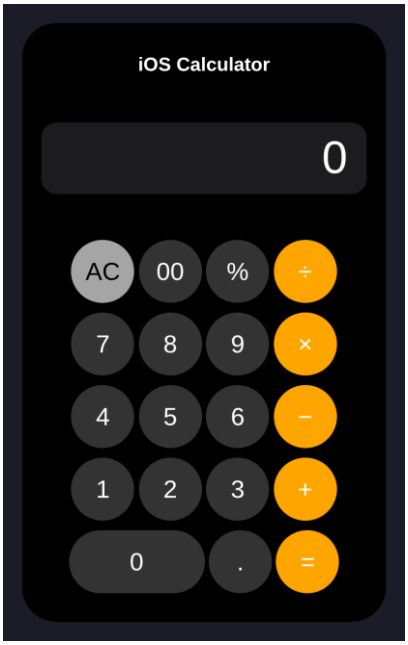


Figure 1: Starting state

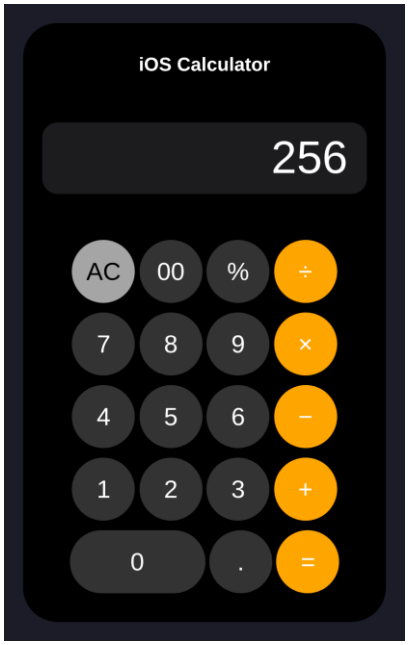


Figure 2: Typing the first number (256)

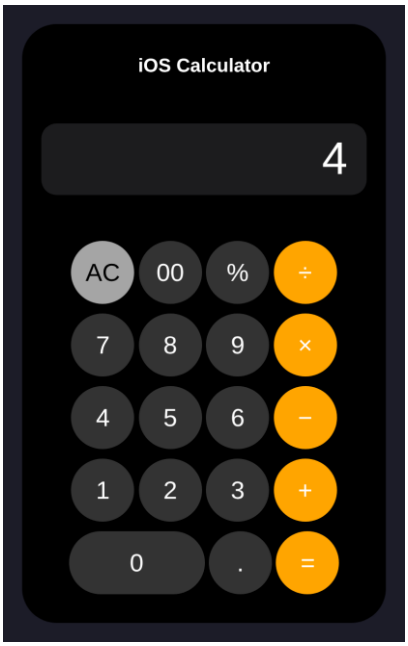


Figure 3: After pressing ×, typing 4

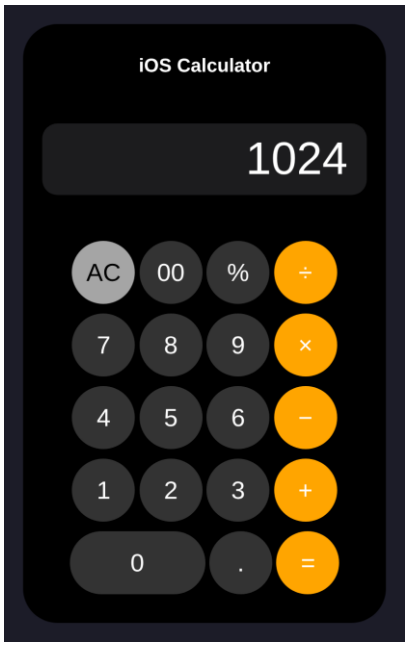


Figure 4: Result (256 × 4 = 1024)

5. Testing

Each case was entered by clicking the buttons, and the result shown matched the expected value.

Input	Expected	Result
12 + 7 =	19	19 ✓
9 × 8 =	72	72 ✓
100 ÷ 4 =	25	25 ✓
5 − 8 =	-3	-3 ✓
1.5 × 2 =	3	3 ✓
50 %	0.5	0.5 ✓
1 00 + 1 =	101	101 ✓
8 ÷ 2 = × 3 =	12	12 ✓
AC	0	0 ✓

6. Source Code

6.1 index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>iOS Calculator</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="calculator">
        <h1>iOS Calculator</h1>

        <p id="display">0</p>

        <div id="keys">
            <div class="row">
                <button class="key light">AC</button>
                <button class="key">00</button>
                <button class="key">%</button>
                <button class="key operator">&divide;</button>
            </div>
            <div class="row">
                <button class="key">7</button>
                <button class="key">8</button>
                <button class="key">9</button>
                <button class="key operator">&times;</button>
            </div>
            <div class="row">
                <button class="key">4</button>
                <button class="key">5</button>
                <button class="key">6</button>
                <button class="key operator">&minus;</button>
            </div>
            <div class="row">
                <button class="key">1</button>
                <button class="key">2</button>
                <button class="key">3</button>
                <button class="key operator">+</button>
            </div>
            <div class="row">
                <button class="key zero">0</button>
                <button class="key">.</button>
                <button class="key operator">=</button>
            </div>
        </div>
    </div>

    <script src="script.js"></script>
</body>
</html>
```

6.2 style.css

```
body {
    background-color: #1c1c28;
    font-family: Arial, sans-serif;
}

.calculator {
    width: 340px;
    margin: 40px auto;
    padding: 20px;
    background-color: black;
    border-radius: 36px;
}
```

```
h1 {
    color: white;
    text-align: center;
    font-size: 20px;
}

#display {
    background-color: #1c1c1e;
    color: white;
    font-size: 48px;
    text-align: right;
    padding: 10px 20px;
    border-radius: 16px;
    overflow: hidden;
}

.row {
    text-align: center;
    margin-bottom: 10px;
}

.key {
    display: inline-block;
    width: 66px;
    height: 66px;
    border: none;
    border-radius: 50%;
    background-color: #333;
    color: white;
    font-size: 26px;
}

.light {
    background-color: #a5a5a5;
    color: black;
}

.operator {
    background-color: orange;
}

.zero {
    width: 142px;
    border-radius: 33px;
}
```

6.3 script.js

```
const display = document.getElementById("display");
const keys = document.getElementById("keys");

let firstNumber = 0;
let operator = "";
let newNumber = true;

function calculate(a, b, op) {
    switch (op) {
        case "+":
            return a + b;
        case "−":
            return a - b;
        case "×":
            return a * b;
        case "÷":
            return a / b;
    }
```

```
}

keys.addEventListener("click", (e) => {
    if (e.target.nodeName !== "BUTTON") {
        return;
    }

    const value = e.target.textContent;

    if (value === "AC") {
        display.textContent = "0";
        firstNumber = 0;
        operator = "";
        newNumber = true;
    } else if (value === "+" || value === "−" || value === "×" || value === "÷") {
        firstNumber = Number(display.textContent);
        operator = value;
        newNumber = true;
    } else if (value === "=") {
        if (operator !== "") {
            const secondNumber = Number(display.textContent);
            display.textContent = calculate(firstNumber, secondNumber, operator);
            operator = "";
            newNumber = true;
        }
    } else if (value === "%") {
        display.textContent = Number(display.textContent) / 100;
    } else if (newNumber) {
        display.textContent = value;
        newNumber = false;
    } else {
        display.textContent = display.textContent + value;
    }
});
```

7. Conclusion

This lab showed how JavaScript makes a webpage interactive. HTML defined the calculator's buttons and screen, CSS gave it the iOS look using inline-block buttons and the box model, and JavaScript selected elements from the DOM, listened for clicks, and used conditionals and a small function to do the calculations and update the screen. Using event delegation kept the code short, since one listener handles all 19 buttons.